

Caching Fundamentals

Phase 1 · Fundamentals · Estimated time: 1.5 hours

1. Concept From First Principles

A **cache** is a small, fast storage layer that keeps a copy of frequently-needed data so future requests can be served without repeating expensive work — usually without querying the primary database or re-running a heavy computation. Caching is one of the highest-leverage tools in system design: a well-placed cache can turn a 200ms database query into a 2ms lookup.

Backend engineer analogy

You've done this exact thing with Laravel's **Cache::remember()** wrapping an expensive query, or Redis storing a rendered API response. System design just asks: where should this cache live (in-process? a shared Redis cluster? a CDN?), and what happens when the underlying data changes?

The three core caching patterns

- **Cache-Aside (Lazy Loading):** the application checks the cache first; on a miss, it reads from the database and *then* writes the result into the cache. Most common pattern — simple, and only caches data that's actually requested.
- **Write-Through:** every write goes to the cache and the database together, synchronously. Keeps cache and database always in sync, but writes are slower since both must succeed.
- **Write-Back (Write-Behind):** writes go to the cache first and are flushed to the database asynchronously later. Fast writes, but risks data loss if the cache fails before flushing.

Eviction: what happens when the cache is full

A cache has finite memory, so it needs a policy for what to discard when full. **LRU (Least Recently Used)** is the most common default — evict whatever hasn't been accessed in the longest time. Other policies include LFU (Least Frequently Used) and simple TTL-based expiry (evict after N seconds regardless of usage).

The classic caching failure mode: stale data

Caching's fundamental risk is serving *outdated* data after the underlying record changes. The usual fixes are setting a sensible TTL (accept some staleness, bounded by time) or actively invalidating (deleting) the cache entry the moment the underlying data changes. This trade-off between freshness and performance is something every interviewer expects you to name explicitly, not gloss over.

It's also worth distinguishing *where* a cache can live, since "caching" isn't one single layer. An **in-process cache** (a variable or local map inside your application's memory) is the fastest possible option but only visible to that one server instance — useless once you have multiple app servers behind a load balancer, since each would cache independently and inconsistently. A **shared cache** like Redis, reachable over the network by every app server, is slightly slower per-lookup but consistent across your

whole fleet — which is why it's the default choice the moment you've scaled past one server (Chapter 12).

2. Real-World Example / Case Study

PhonePe and Razorpay-style payment platforms cache things like merchant configuration and rate-limit counters in Redis because these are read constantly but change rarely — a perfect cache-aside candidate. Meesho and Flipkart cache product listing pages and search results heavily, since the same popular products are viewed by millions of users who all see essentially the same data — caching here turns a repeated, expensive catalog query into a shared, cheap Redis read.

A frequently cited real incident pattern (and a great interview story) is **cache stampede**: a extremely popular cache key (e.g., a flash-sale product) expires, and thousands of concurrent requests all miss the cache at once and hammer the database simultaneously, sometimes taking it down. Mitigations include locking (only one request repopulates the cache while others wait) or staggered/jittered TTLs so keys don't all expire at the exact same instant.

A second example: Meesho caches category-level product listings aggressively since these change relatively infrequently (a few times a day, driven by merchandising updates) but are read by nearly every user who opens the app — an extremely favorable read:write ratio that makes even a fairly long TTL (minutes, not seconds) an easy, safe win with minimal staleness risk.

3. Diagram

Cache-Aside (most common pattern):

```
App --- 1. GET product:123 --> Cache
App <-- 2. MISS ----- Cache
App --- 3. SELECT * FROM products WHERE id=123 --> Database
App <-- 4. row returned ----- Database
App --- 5. SET product:123 = row, TTL 300s -----> Cache
App returns row to caller
```

Next request for product:123 within 300s:

```
App --- GET product:123 --> Cache
App <-- HIT (fast!) ----- Cache
```

One last practical point: caching only helps if the same data is requested repeatedly — it does nothing for genuinely unique, one-off lookups, and can even add overhead (a wasted cache write for data that's never read again) if applied blindly everywhere. Before adding a cache, it's worth confirming there actually is a "hot" subset of data or queries being repeated; this is exactly the kind of judgment call your back-of-the-envelope estimation skills (Chapter 14) help you make with real numbers instead of a guess.

4. Trade-offs Table

Pattern	Read speed after warm-up	Write consistency	Risk
Cache-Aside	Fast (after first miss)	Cache can go stale until TTL/invalidation	Cache stampede on hot key expiry

Pattern	Read speed after warm-up	Write consistency	Risk
Write-Through	Fast, always fresh	Strong — cache and DB always match	Every write pays double latency
Write-Back	Fast reads and writes	Weak — risk of data loss before flush	Data loss if cache crashes before flush

5. Common Interview Questions

Q1: "How would you keep a cache from serving stale data?"

Model approach: Offer two levers — a bounded TTL (accept staleness up to N seconds, simple, works for most read-heavy data) or active invalidation (delete/update the cache entry the instant the DB write happens, better freshness but more moving parts and possible race conditions to reason about).

Q2: "A flash-sale product page just crashed your database. What happened, and how do you prevent it next time?"

Model approach: Name cache stampede explicitly — a hot key expired and thousands of concurrent requests all missed the cache simultaneously and hit the DB at once. Propose a locking mechanism (only one request repopulates the cache, others wait briefly) or pre-warming the cache with a long TTL before the sale starts.

Q3: "When would an in-process (local) cache still make sense, even in a multi-server system?"

Model approach: For data that's extremely hot, tiny, and tolerant of slight per-instance inconsistency (e.g., feature flags, small config values), a local in-process cache avoids even the small network hop to Redis, layered on top of a shared cache as an L1/L2 combination. This is an optimization, not a default — mention you'd only reach for it after confirming the shared cache alone isn't fast enough.

6. Practice Exercises

- 1 For a news-feed "like count" that updates constantly, decide between cache-aside, write-through, and write-back, and justify it.
- 2 Explain cache stampede to someone who has never heard the term, using a real-world (non-technical) analogy.
- 3 Design the cache key naming scheme for a product catalog cache (e.g., how would you key it, and how would you invalidate it when a price changes?).

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"Redis Caching Strategies Explained: Cache Aside vs Write-Through Patterns"	Directly compares the patterns covered in this chapter.
English	"Caching Explained: Redis, Cache-Aside, & LRU" (System Design Tutorial)	Ties caching directly to system design interview framing.

Language	Video & Channel	Why it helps
Hindi	Redis explained in Hinglish (covers caching, rate limiting, sessions, OTP use cases)	Practical Hindi/Hinglish walkthrough of real Redis use cases.
Hindi	"Redis Full Course 2026 Caching, Rate Limiting, Pub/Sub" — Chai aur Code	Full Hindi course covering caching patterns in depth.

8. Key Takeaways

- Caching stores frequently-needed data in a fast layer to avoid repeating expensive DB queries or computation.
- Cache-aside is the default pattern: check cache, on miss read DB and populate cache.
- Write-through keeps cache and DB always in sync but slows writes; write-back is fast but risks data loss.
- LRU is the most common eviction policy for a full cache.
- Every cache introduces a staleness trade-off — you must explicitly decide how stale is acceptable.
- Cache stampede (a hot key expiring under massive concurrent load) is a classic real-world failure and a strong interview talking point.